

CHAPTER 4 — LINEAR SEARCH WITH ARRAYS

Learning Goal

By the end of this chapter, students should be able to use an **array**, **index**, **loop**, and **target value** together to perform Linear Search in C.

SLIDE 1 — Connecting Linear Search with Arrays

What happens when our data is in an array?

An array stores multiple values under one name.

```
int numbers[] = {10, 25, 40, 15, 30};
```

Think of it like a row of boxes .

```
[10] [25] [40] [15] [30]
```

```
0 1 2 3 4 ← Index
```

If we want to find 15:

 **Target = 15**

The program checks:

```
numbers[0] → 10 ❌
```

```
numbers[1] → 25 ❌
```

```
numbers[2] → 40 ❌
```

```
numbers[3] → 15 ✅
```

 **Memory Trick**

Array = Values + Index

 **Remember:** Linear Search checks the array **one element at a time**.

SLIDE 2 — Using Index to Check Each Element

 **How does C access each value?**

C uses an **index** to access an array element.

numbers[i]

Here:

- **numbers** → array name 
- **i** → current index 
- **numbers[i]** → value at that index 

Example:

```
int numbers[] = {8, 12, 20, 25};
```

Index	Value
-------	-------

0	8
---	---

1	12
---	----

2	20
---	----

3	25
---	----

If:

i = 2;

then:

numbers[i]

means:

numbers[2] → 20

💡 **Memory Trick**

i tells WHERE → numbers[i] tells WHAT

🟪 SLIDE 3 — Searching an Array Using a Loop

🔄 **Let the loop do the repeated checking!**

Instead of writing:

```
if(numbers[0] == target)
if(numbers[1] == target)
if(numbers[2] == target)
```

We can use a **for** loop:

```
for(int i = 0; i < 5; i++)
{
    if(numbers[i] == target)
    {
        printf("Found");
        break;
    }
}
```

🧠 **What is happening?**

1	2
3	4

i = 0 → Start from first element



🔍 **numbers[i]** → Check current element



✗ Not equal → `i++` → Move forward



✓ Equal → Target found → `break`

🎯 Visual Flow

Start → Check → Move → Check → Move → Found!



Memory Trick

Loop controls WHERE you search.

🟡 SLIDE 4 — Complete Linear Search Program

💻 Let's put everything together!

```
#include <stdio.h>
```

```
int main()
{
    int numbers[] = {10, 25, 40, 15, 30};
    int target = 15;

    for(int i = 0; i < 5; i++)
    {
        if(numbers[i] == target)
        {
            printf("Found");
            break;
        }
    }

    return 0;
}
```

What is happening?

 **Array** → stores the values

 **target** → value we want to find

 **for loop** → moves through the array

 **i** → keeps track of the position

 **numbers[i]** → gets the current value

 **break** → stops after finding the target

Quick Memory

Array + Target + Loop + Index = Linear Search in C

SLIDE 5 — Trace the Search Through an Array

 Let's watch the program work!

Array:

[10] [25] [40] [15] [30]
0 1 2 3 4

 Target = 15

Step	i	number s[i]	Match?
1	0	10	✗
2	1	25	✗
3	2	40	✗
4	3	15	✓ FOUND!

🚦 Once 15 is found:

break;

stops the loop.

💡 **BIG MEMORY TRICK**

 **i = Position**

 **numbers[i] = Value**

 **target = What we are searching for**

🏆 **LEVEL CLEARED!**

CHAPTER 4 — QUIZ

● **Q1 — Fill in the Code**

Complete the missing part:

```
int numbers[] = {10, 20, 30, 40};
```

```
int target = 30;
```

```
for(int i = 0; i < 4; i++)
```

```
{
```

```
    if(_____ == target)
```

```
    {
```

```
        printf("Found");
```

```
        break;
```

```
    }
```

}

 **Hint:** Use the array name and current index to access the element.

Glossary

-  **Index** — Position of an element in an array.
-  **Array** — Collection of values stored under one name.

✓ **Answer:** `numbers[i]`

● Q2 — Fill in the Code

Complete the loop:

```
int numbers[] = {5, 15, 25, 35, 45};
```

```
for(int i = 0; i < _____; i++)  
{  
    printf("%d ", numbers[i]);  
}
```

 **Hint:** The array contains **5 elements**.

Glossary

-  **Loop** — Repeats a set of statements.
-  **Element** — A single value stored in an array.

✓ **Answer:** `5`

● Q3 — MCQ with Code

What will be printed?

```
int numbers[] = {10, 20, 30, 40};
```

```
int target = 30;

for(int i = 0; i < 4; i++)
{
    if(numbers[i] == target)
    {
        printf("Found");
        break;
    }
}
```

- A) 10
- B) 20
- C) Found
- D) Nothing

 **Hint:** Which array element matches the target?

Glossary

-  **Target** — The value we want to find.
-  **Match** — When the current value is equal to the target.

 **Answer:** C) Found

Q4 — Basic MCQ

In the expression:

numbers[i]

what does **i** represent?

- A) The value being searched
- B) The array name

- C) The index of the current element
- D) The size of the array

 **Hint:** Think about how `i` changes inside the loop.

Glossary

-  **Index** — Position used to access an element in an array.
-  **Element** — A value stored inside an array.

 **Answer: C) The index of the current element**

Q5 — Fill the Missing Code Line

Complete the code:

```
if(numbers[i] == target)
{
    printf("Found");
    _____;
}
```

 **Hint:** We don't need to continue searching after finding the target.

Glossary

-  **break** — Stops the loop immediately.
-  **Search** — Process of finding a required value.

 **Answer: `break`**

 **CH 4 QUIZ CLEARED!**

Array → Index → Loop → Target → Search 